

COMPREHENSIVE PROJECT & PROCEDURAL DOCUMENT

Workstation Environment Baseline, Deployment Parameters, and Operations Registry

1. Executive Summary

This comprehensive document consolidates all project information, system architectures, hardware boundaries, and deployment guidelines into a single, structured master file. By mapping foundational environment state matrices, removing historical version conflicts, and applying production-grade configuration definitions, this registry acts as the sole source of truth for the system workstation. It outlines standard operating procedures to maintain long-term infrastructure stability and clear, deterministic deployment boundaries.

2. Document Revision Procedures

To preserve configuration integrity across subsequent maintenance lifecycles, all future updates, additions, and documentation changes must strictly adhere to the following three-stage operational workflow:

Standard Documentation Workflow

- Drafting Stage:** Submit all raw environment data, shell script revisions, hardware changes, and administrative modifications directly through the secure deployment interface to generate a preliminary technical draft.
- Refining Stage:** Execute a manual validation review against the primary document constraints. All non-standard jargon, hypothetical feature behaviors, and ambiguous platform designations must be eliminated to ensure idiomatic, precise engineering language.
- Exporting Stage:** Generate the finalized version as a structurally sound, well-formatted PDF within the distribution environment for immediate digital signing, archival, and system deployment tracking.

3. Exhaustive Data Compilation

The following sections compile all verified workstation specifications, graphics driver constraints, verification utilities, and subsystem configurations into a centralized, easily referenced operational record.

3.1 System Specifications & Hardware Profile

Subsystem Component	Hardware Specification Details	Administrative and Operational Consequences
Motherboard	ASUS TUF X299 Mark 1	Fixed PCIe lane topology allocation: Slot 1 operates at full x16 width; Slot 3 operates at x8 width; Slot 4 is forced to x1 width due to motherboard lane sharing. Intel Rapid Storage Technology (RST) couples the M.2_1 slot, the PCIeX4_2 slot, and SATA ports 5 through 8, binding mutual device availability.
Processor (CPU)	Intel Core i9-10900X (Cascade Lake-X)	10 physical cores, 20 execution threads, 48 native CPU PCIe lanes. Features dual AVX-512 FMA blocks. Requires explicit governor, turbo boost, and kernel mitigation configuration. AVX-512 instructions dictate specialized compilation flags for machine learning workloads.
Volatile Memory (RAM)	156.9 GiB DDR4-2933	Quad-channel memory architecture. Provides significant capacity headroom for large tmpfs mounts, aggressive kernel zram memory compression allocation, and deep virtual memory (vm.*) kernel tuning.
Dual Graphics (GPUs)	2× NVIDIA TITAN Xp Collectors Edition	Pascal architecture (GP102 core). Hardware is strictly frozen to the legacy R580 proprietary driver branch. This represents the absolute architectural ceiling for display server stability and compute libraries.
Audio Subsystem	Native Instruments Komplete Audio 6	External USB audio interface. Demands dedicated real-time priority mapping and device routing rules within the PipeWire sound server topology.
Network Interfaces	Intel I219-V + Intel I211	Dual onboard Gigabit Ethernet controllers. Subject to deterministic udev persistent device naming rules, mapped directly to standard NetworkManager connection profiles. One interface remains active while the backup link is held down.
Thermal Management	NZXT Kraken All-in-One (AIO)	Liquid cooling infrastructure. Pump speed, fan loops, and thermal telemetry are

Subsystem Component	Hardware Specification Details	Administrative and Operational Consequences
		monitored via custom kernel modules and exposed via the standard lm-sensors framework.
Non-Volatile Storage	4-Drive Physical Drive Layout	The root operating system and primary system boundaries are mapped entirely to the fourth physical storage drive (/dev/sdd).

3.2 Operating System Baseline & Version Tiers

An administrative environment evaluation must be performed prior to running automation scripts to determine which desktop tier is active on the machine:

- **Distribution Baseline:** Kubuntu 24.04 LTS (Noble Numbat), running the long-term supported Ubuntu base.
- **Stock Desktop Specifications:** KDE Plasma 5.27.11, KWin 5.27.11, KDE Frameworks 5.x, and Qt 5.
- **Stock Session Protocol:** Natively defaults to X11. While the plasma-workspace-wayland package is present in the archive as an option, it lacks official distribution support on the Plasma 5 baseline.
- **Kernel Architecture:** Shipped with the stock 6.8 General Availability (GA) kernel. Subsequent deployment of Hardware Enablement (HWE) point releases will advance this version; kernel versioning must be dynamically queried at runtime.
- **Plasma 6 Availability:** Accessible solely through the manual activation of the official Kubuntu Backports PPA repository. This upgrade replaces core system libraries with Qt 6 and KDE Frameworks 6, an architectural choice that changes configuration utilities and display mechanics.

3.3 NVIDIA Pascal Hardware & Driver Policy Constraints

Because the dual NVIDIA TITAN Xp graphics cards utilize the legacy Pascal (GP102) architecture, the R580 driver tree is the final branch to support these cards. The terminal driver release version for this machine is strictly bounded to the **580.105.08** series or its immediate security point updates. This limitation imposes absolute constraints across all areas of this workstation:

1. **Apt Package Pinning:** The graphics driver must be locked to the 580 legacy branch using strict package manager pinning. Upgrades to the 590+ branches will break hardware acceleration and disable video output.
2. **Proprietary Module Requirement:** Open-source NVIDIA kernel modules (-open flavors) require a hardware-based Graphics System Processor (GSP) engine, which is only present on Turing architecture cards or newer. Pascal GPUs lack a GSP; therefore, the system must run the fully closed, proprietary binary kernel module (nvidia-driver-580).

3. **CUDA Library Cap:** CUDA 12.x is the final major runtime framework version that supports Pascal architecture cards. CUDA 13 completely removes support for GP102 silicon. All isolated development environments, Python virtual environments, and virtual runtimes must be resolved against a CUDA-12-compliant toolkit.
4. **Display Protocol Stability Bias:** Dual Pascal GPUs running on a frozen legacy driver branch present structural edge cases under Wayland. While Wayland is functional when using the Plasma 6 Backports tier, the X11 display server provides significantly lower risk, fewer compositing anomalies, and higher stability for this hardware generation. On the stock Plasma 5.27 base, X11 is mandatory.
5. **Compute Co-Processing Configuration:** Scalable Link Interface (SLI) is entirely disabled for computational tasks. The two GP102 graphics devices operate as completely independent CUDA endpoints. Low-level management is performed via the `nvidia-persistenced` daemon alongside explicit per-device power and clock thresholds set via the `nvidia-smi` utility.
6. **Kernel Update Risks:**** Post-End-of-Life (EOL) legacy drivers receive minimal software maintenance, limited strictly to critical security patches or basic compatibility fixes for newer Linux kernels. Every upstream Hardware Enablement (HWE) kernel upgrade must be carefully validated against the locked 580 DKMS source tree prior to system reboot.

3.4 Automated Workstation Environment Audit Script

The following production shell script provides automated runtime verification of active software versions, driver constraints, display servers, and system configuration profiles. This utility dynamically resolves active configuration paths:

```
#!/bin/bash
# =====
# WORKSTATION BASELINE AUDIT SCRIPT
# Verification tool for Kubuntu 24.04 LTS with Legacy NVIDIA GPUs
# =====

set -u

echo "=== [1/5] OPERATING SYSTEM & KERNEL BASELINE ==="
if [ -f /etc/os-release ]; then
    lsb_release -ds
else
    echo "Unknown Linux Distribution"
fi
echo "Kernel Release Version: $(uname -r)"
echo "Active Session Protocol: ${XDG_SESSION_TYPE:-Unknown Display Protocol}"
echo "Current Desktop Workspace: ${XDG_CURRENT_DESKTOP:-Unknown Desktop Environment}"

if command -v plasmashell >/dev/null 2>&1; then
    echo "KDE Plasma Version: $(plasmashell --version)"
else
    echo "KDE Plasma: Not found in system path"
fi

echo -e "
=== [2/5] REPOSITORY PROFILE & PLASMA 6 AVAILABILITY ==="
if ls /etc/apt/sources.list.d/ | grep -qi "backports"; then
    echo "Kubuntu Backports PPA Status: ENABLED (Plasma 6 Tier Active)"
else
    echo "Kubuntu Backports PPA Status: NOT DETECTED (Stock Plasma 5.27 Tier Active)"
fi

echo -e "
=== [3/5] NVIDIA HARDWARE STATUS & DRIVER PINNING ==="
if command -v nvidia-smi >/dev/null 2>&1; then
    nvidia-smi --query-gpu=index,name,driver_version,memory.total --format=csv
else
    echo "CRITICAL: nvidia-smi utility is not responding or driver is missing"
fi

if command -v apt-mark >/dev/null 2>&1; then
    HELD_PACKAGES=$(apt-mark showhold | grep -i "nvidia")
    if [ -n "$HELD_PACKAGES" ]; then
        echo "Apt Package Pins Detected: $HELD_PACKAGES"
    else
        echo "WARNING: NVIDIA driver packages are NOT pinned via apt-mark. Fix
immediately."
    fi
fi
```

3.5 System Subsystem Configurations

3.5.1 Boot Loader, Initramfs, and Core Kernel Command Line

Configures standard UEFI boot order parameters, Compatibility Support Module (CSM) exclusion, Resizable BAR mapping, and Above-4G Decoding to ensure correct MMIO memory layout for the dual GPUs. Manages Secure Boot parameters and Machine Owner Keys utilizing `mokutil` to ensure valid signing of the proprietary closed-source NVIDIA driver modules.

GRUB environment boot configuration (`/etc/default/grub`) enforces mandatory boot flags, including `nvidia-drm.modeset=1` to ensure early kernel modesetting, specific `iommu` configuration parameters, explicit CPU hardware mitigations configurations, and `pcie_aspm=off` flags to maintain maximum PCIe bus stability. Initial Ramdisk optimization (`initramfs-tools`) hardcodes the proprietary NVIDIA kernel modules directly into the primary initramfs image alongside basic LUKS cryptsetup and LVM parameters to prevent display blanking or race conditions during boot. Early microcode loading is explicitly enforced via `intel-microcode`.

3.5.2 Kernel Subsystems, Hardware Control, and Thermal Telemetry

Configures `/etc/modprobe.d/` rule structures to enforce custom proprietary driver behavioral properties, complete blacklisting of the open-source nouveau driver, and custom module signature checking routines under Secure Boot. Implements fine-grained kernel subsystem performance definitions via `/etc/sysctl.d/` to adjust `inotify` watches, socket performance limits, and virtual memory page writeback behaviors.

Adjusts hardware execution states, turbo limits, and process boundaries using `intel_pstate` drivers and explicit core affinity tables for heavy AVX-512 compilation runs. Enforces persistent memory initialization using the `nvidia-persistenced` daemon. Sets absolute maximum power targets and clock rate limits across both separate physical GP102 GPUs via automated `nvidia-smi` service startup commands. Uses custom kernel configurations to pull internal telemetry from the NZXT liquid cooling loop and system fans, exposing raw instrumentation metrics to user space via the standard `lm-sensors` API framework.

3.5.3 System Services, State Management, and Initialization

Manages core system units, user-level service declarations, target sequences (such as `graphical.target`), drop-in directory parameter additions, and performance diagnostic tuning via `systemd-analyze`. Coordinates hard physical power events, lid actions, and systemic timeout thresholds within `logind.conf`. Every physical event must be explicitly allocated to an owner to prevent conflicting actions with the user-space PowerDevil environment daemon. Sets data limits and persistent logging intervals for the system logging framework (`journal`). Handles precise network time protocols via `systemd-timesyncd` or localized `chrony` configurations. Directs nameserver routing policies via `systemd-resolved` integrated with NetworkManager.

3.5.4 Storage Topologies, Filesystems, and Mount Specifications

Configures standard GUID Partition Tables (GPT) using the parted utility, manages logical storage spans using LVM, and applies custom tuning variables for Ext4, XFS, or Btrfs. Implements system mounting rules within /etc/fstab including noatime parameters, automated SSD block trim scheduling (fstrim.timer), full cryptographic block device mappings via LUKS (/etc/crypttab), and physical drive arrays combined under Intel RST firmware definitions.

If a storage partition undergoes an emergency conversion to read-only status due to an unhandled kernel error, perform the following troubleshooting workflow to isolate the root cause before attempting data recovery:

```
# 1. Inspect active mount statuses and flags
findmnt
# 2. Extract explicit read-only mount parameters
mount | grep ' ro '
# 3. Locate active processes pinning the volume
lsof +D /path/to/target/mount
# 4. Read underlying physical block device lock states
lsblk -o NAME,RO
blockdev --getro /dev/sdX
```

3.5.5 Identity Management, Authentication, and Security Posture

Defines persistent user identities, local group boundaries (video, render, sudo, plugdev), and access parameters. Manages login parameters, security stacks, session environments, and resource consumption boundaries via configuration definitions located in /etc/pam.d/ and limits.conf. Mandates precise user escalation authorization schemas via policy definitions within /etc/sudoers.d/ and system-wide Polkit configurations (polkit-1/rules.d/). Enforces security policies using standard ufw or raw underlying nftables implementations. Mandates AppArmor profiling configurations to isolate critical system apps. Configures server-side and client-side secure shell parameters (sshd_config). Handles automated decryption of user credentials via standard PAM-to-KWallet secure interface mapping.

3.5.6 Package Management, Repository Pinning, and Development Toolchains

Governs basic dpkg routines, customized repository routing files inside /etc/apt/sources.list.d/, and cryptographic archive keys inside /etc/apt/keyrings/. Controls the core system state transition via the Kubuntu Backports PPA. Activating the Backports PPA migrates the host environment to KDE Plasma 6; this process must be treated as a one-way architectural choice due to database updates and library dependencies.

To protect the legacy driver tree from accidental runtime upgrades, create /etc/apt/preferences.d/nvidia-580-pin with the following contents:


```
Package: nvidia-driver-* nvidia-dkms-* libnvidia-*  
Pin: version 580*  
Pin-Priority: 1001
```

Implements a strict version ceiling for machine learning stacks, restricting packages to a CUDA 12.x framework version. Implements complete isolation models for local application development. Strictly forbids system-wide Python changes via pip or the use of the `--break-system-packages` flag, ensuring total compliance with PEP 668 policies. All localized development environments must run under isolated uv toolchain boundaries or standard virtual environments (venv) placed under `~/envs/`.

3.5.7 Display Servers, Graphical Desktop Sessions, and Desktop Portals

Controls display server selection behaviors. Under the Stock tier, the workstation runs on an X11 server environment. Under the Backports-6 tier, both servers become available; X11 provides maximum stability for dual Pascal cards, while Wayland enables modern window management features and portal integration. Manages physical workspace arrangements, resolution scaling rules, and frame mapping properties via the standard KScreen configuration system (`~/.local/share/kscreen/` or `kscreen-doctor`). Under Stock X11 environments, forces interface uniformity using global font DPI adjustments. Under the Backports-6 Wayland tier, uses native per-output fractional display handling with grid pixel alignment.

The desktop portal framework manages the broker application layer running above the window compositor and below independent user applications. Under Wayland sessions, this component acts as the primary access control boundary for system sharing, application input tracking, and cross-application communication. Operates using a core system communications manager (`xdg-desktop-portal`) combined with a specialized user interface bridge backend (`xdg-desktop-portal-kde`).

- **System Video Stream Sharing:** Handles screen capture demands, desktop recording tools, and network interface sharing utilities via dedicated PipeWire pipelines. This ensures media tools operate correctly with legacy NVIDIA drivers.
- **File Picker Integration:** Overrides isolated sandbox environments to present native KDE file choice screens inside Flatpak or Snap applications.
- **Input Grabbing Systems:** Captures system-wide key bindings and coordinates synthetic pointer positioning actions for automation utilities under Wayland display protocols.

3.5.8 Window Management, UI Customization, and Application Integration

Dictates custom application styles, specific window constraints, virtual workspace loops, and system-wide tiling behaviors via parameters located inside `kwinrc` and `kwinrulesrc`. Triggers live runtime updates to the window manager without restarting the graphical session via the DBus connection parameter: `qdbus org.kde.KWin /KWin reconfigure`. Manages desktop widget appearances, specific application layouts, and custom palette definitions using base variables written into the shared `kdeglobals` configuration file. Coordinates uniform display styling definitions for GTK applications using the configuration module helper

(kde-gtk-config). Controls the Baloo system background storage indexer; if disk I/O performance degrades during massive data writes, disable the engine using `balooctl suspend`.

To ensure local web applications, dashboards, or local service interfaces behave identically to native system tools, deploy a localized application definition file inside `~/.local/share/applications/local-web-app.desktop`:

```
[Desktop Entry]
Type=Application
Name=Local Application Interface
Comment=Production Local Service Dashboard
Exec=google-chrome --app=http://localhost:8080/ --user-data-dir=%h/.local/share/webapps/dashboard
Icon=utilities-terminal
Categories=Development;Network;
Keywords=local;dashboard;service;
StartupWMClass=localhost__8080
StartupNotify=true
```

Register the launcher file using: `update-desktop-database ~/.local/share/applications`. The `StartupWMClass` property must precisely match the string generated by the underlying browser engine. This ensures KWin correctly applies window management parameters, groups taskbar elements, and routes custom rules.

3.6 Configuration Element Reference Matrix

Reference Category	Primary Administrative Utilities	Operational Scope	Target Configuration Infrastructure
Boot & Low-Level Initialization	grub-mkconfig, mokutil, update-initramfs	System-Wide	UEFI address structures, kernel parameters, Secure Boot keys
Kernel & Hardware Subsystems	sysctl, modprobe, nvidia-smi, lm-sensors	System-Wide	Kernel options, hardware capabilities, power constraints, telemetry
System Infrastructure	systemctl, journalctl, logind.conf	System-Wide / User	Background services, system targets, timeout hooks
Storage & Security Policies	cryptsetup, parted, ufw, polkit	System-Wide	Storage volumes, encryption rules,

Reference Category	Primary Administrative Utilities	Operational Scope	Target Configuration Infrastructure
			firewall policies, privilege limits
Package Management	apt, apt-mark, flatpak, uv	System-Wide / User	Repository routing, driver locks, development toolchain tracking
Graphics & Desktop Session	kscreen-doctor, xdg-desktop-portal	User Session	Display server protocol, screen layouts, app permissions
Desktop Workspace Environment**	kwriteconfig, qdbus, kpackagetool	User Session	Desktop layouts, window manager behavior, application configurations
Custom Web Integrations	update-desktop-database	User Session	Isolated web application endpoints, custom launcher properties